SIMATS ENGINEERING

DEPARTMENT OF INFORMATION TECHNOLOGY

NATURAL LANGUAGE PROCESSING

ASSIGNMENT 5 – CO5

Language Generation and Discourse Analysis

Student details	Fill in
Student Name	V.Lakshmi Prasad
Register Number	192425179
Department / Section	AI&ML
Faculty Name	Dr Ajitha Rajagopal
Academic Year / Semester	Sep–Dec 2026
Date of Submission	_____

Nature: Individual Duration: One Week Total Marks: 50

Submission note: This document provides a worked reference solution, formal pseudocode, hand traces, test designs and a reproducible Python implementation outline. Replace the clearly labelled illustrative experimental measurements with results from your own execution before submission.

CONTENTS

- PART A – Design, Specification and Pseudocode
- A1. Discourse analyst's world and measurement model
- A2. Discourse units, graph and next-mention
- A3. Generator with focus memory
- A4. Entity–discourse graph and parser repair
- A5. Coherence and global salience
- PART B – Implementation and Investigation
- B1. Discourse analyser implementation and tests
- B2. Adversarial input
- B3. Dialogue acts and conversational agent
- B4. Translation-service engineering decision
- Appendix – Python reference implementation and viva points

PART A – DESIGN, SPECIFICATION AND PSEUDOCODE

Common report T used throughout this solution:

- (1) The department bought a new server for the lab last month.
- (2) It replaced an old machine that had failed twice.
- (3) Ravi configured the server with the lab's toolchain.
- (4) He then installed the compiler suite on it.
- (5) The suite crashed when the memory filled during a build.
- (6) Ravi reported the fault to the vendor, and the vendor replaced a faulty module.

Gold reference links: (2) It → server introduced in (1); (4) He → Ravi in (3); (4) it → server in (1)/(3); (5) The suite → compiler suite in (4). The final link is a definite noun phrase rather than a pronoun, but it is included as a coreference link.

A1 – DEFINE THE DISCOURSE ANALYST'S WORLD (4 MARKS)

A1.1 Position and responsibilities

Pipeline position: raw document → sentence/clause segmentation → lexical and syntactic analysis → discourse analysis/reference resolution → discourse representation → question answering or discourse planning → surface realisation → translation/output.

Module	Receives	Must preserve	May change
Discourse analyser	Sentences, mentions, syntax, entity candidates and context	Who did what to whom; uncertainty; temporal and causal relations	Adds entity links, discourse units and relation labels; does not rewrite facts
Language generator	Meaning representation, focus state and audience constraints	Propositional meaning, entity identity, polarity and event order	Chooses names/pronouns, sentence order where safe, aggregation and wording

The discourse analyser interprets existing text. The generator creates new text from a meaning representation. A generator must not change the underlying facts merely to make the sentence shorter.

Design issue	Example in this assignment	Settled by
Unit of analysis	Sentence is the initial unit; clause split is needed for "Ravi reported…, and the vendor replaced…"	Corpus annotation scheme defines gold boundaries; designer chooses operational segmentation
Knowledge source	"He" agrees with Ravi in number/person; recency and grammatical role help	Designer combines syntax, agreement and salience; corpus supplies gold links
Explicit / implicit relation	"then" signals sequence; crash after memory fills suggests cause	Annotation scheme defines relation labels; designer infers implicit links conservatively
Discourse granularity	Sentence units with clause-level	Designer, subject to annotation

	subunits for coordinated events	policy
Evaluation	Precision, recall and F1 over annotated mention links	Designer defines markables; gold annotation fixes correct answers
Genuine ambiguity	"It" could refer to server or another inanimate machine	System should preserve multiple candidates or abstain; gold annotators record intended reading

A1.2 Measurement model

A markable is a referring expression that denotes an entity: proper names, definite descriptions and referential pronouns. Exclude non-referential "it" (for example, "It is raining") and generic uses. A proposed link is correct only if both the mention and its antecedent match the gold entity chain.

Gold links = 4. System X proposes 5 links, 3 correct: TP=3, FP=2, FN=1. Precision=3/5=0.60; Recall=3/4=0.75; F1=2×0.60×0.75/(0.60+0.75)=0.667. System Y proposes 4 links, 4 correct: TP=4, FP=0, FN=0; Precision=1.00, Recall=1.00, F1=1.00.

System	TP	FP	FN	Precision	Recall	F1
X	3	2	1	0.600	0.750	0.667
Y	4	0	0	1.000	1.000	1.000

Limits: (1) the gold interpretation may itself be ambiguous; (2) link-level F1 does not measure whether the final generated paragraph is coherent or factually adequate.

A1.3 Reference-resolution pseudocode

```
ALGORITHM RESOLVE(mention m, candidates C, context)
INPUT: mention m; candidate entities C; discourse context
OUTPUT: antecedent entity or UNRESOLVED

1. C1 ← candidates introduced before m and within the active discourse segment
2. C2 ← filter C1 by number, person and (when reliable) gender agreement
3. C3 ← filter C2 by syntactic constraints and semantic type compatibility
4. IF C3 is empty THEN RETURN UNRESOLVED
5. Rank each candidate by:
     grammatical role (subject > object > oblique),
     recency (nearer > farther),
     repeated mention / topic continuity
6. IF top two candidates have equal or near-equal scores
     THEN RETURN AMBIGUOUS(C3)
7. IF top candidate violates a hard constraint THEN RETURN UNRESOLVED
8. RETURN highest-ranked candidate
END
```

Fallbacks: keep an explicit ambiguous set; if no candidate survives, mark unresolved; if confidence is below a threshold, ask for clarification or retain the original expression instead of inventing a link.

A1.4 Candidate trace for "it" in T(4)

Candidate	Agreement	Semantic fit	Recency / role	Decision
server (T1/T3)	Compatible: singular, inanimate	Can host software	Recent; object in T3	Strong candidate
compiler suite (T4)	Same sentence; singular, inanimate	"installed … on it" cannot mean suite in this reading	Very recent but semantic role mismatch	Reject
old machine (T2)	Compatible	Could host software	Less recent; not current topic	Lower score

The weak spot is that recency alone would choose the closest noun phrase, even when semantic role and event structure make it implausible. The resolver must combine constraints rather than use a single nearest-noun rule.

A2 – UNITS, DISCOURSE GRAPH AND NEXT-MENTION (5 MARKS)

A2.1 Boundary rules

- Start a new unit at a sentence boundary; split coordinated clauses only when each has an independent predicate and discourse contribution.
- Do not split at a connective inside a clause merely because a cue word occurs; "and" can coordinate predicates without creating an independent discourse segment.
- Treat paragraph and section boundaries as strong topic-boundary signals, but retain cross-boundary entity links when explicitly supported.
- Quoted speech can form a nested speaker-attributed unit; do not merge the speaker's claims with the narrator's commitments.

A2.2 Segmentation and discourse structure

For T, use six sentence units U1–U6. U1 establishes the purchase; U2 elaborates the server replacement; U3 advances the configuration event; U4 continues the installation sequence; U5 gives the failure/result; U6 reports the fault and vendor repair. Relations: U2–U1 = Elaboration; U3–U2 = Sequence; U4–U3 = Sequence; U5–U4 = Result/Cause (the crash follows memory exhaustion); U6–U5 = Sequence with an implicit problem-response relation.

```
ALGORITHM SEGMENT_AND_LINK(sentences)
1. Create one EDU-like unit for each sentence.
2. Split a sentence at a coordinated clause only if both sides contain an independent predicate.
3. For each new unit u:
     detect explicit cue phrases;
     otherwise infer a relation only when lexical, temporal or causal evidence supports it;
```

```
    attach u to the nearest compatible open unit on the right frontier.
4. Record relation type, nucleus, satellite, predecessor and successors.
5. Close units that are no longer on the right frontier.
END
```

Nucleus = the central claim; satellite = supporting or elaborating material. In U4→U3, the installation event is the nucleus and temporal continuation is the relation. In U5, the crash is the main event and memory filling is the causal circumstance. The right-frontier constraint prevents attaching a new unit to a structurally closed unit without reopening it.

A2.3 Backward next-mention scan

Scan mentions from the end towards the beginning. Maintain NEXT[entity], the nearest later mention position. At each mention, record whether a later mention exists and its distance, then update NEXT. For accessibility, use entities on the active right frontier; entities that never appear again are not predicted as next mentions.

Unit / mention	Entity	Next later mention	Distance in units	Available on exit?
U2: It	server	U3: the server	1	Yes
U2: old machine	old_machine	No later mention	—	No
U3: Ravi	Ravi	U4: He	1	Yes
U3: the server	server	U4: it	1	Yes
U3: toolchain	toolchain	No later mention	—	No

A forward scan cannot know the next mention until it has already passed it; it therefore measures previous mentions, not next-mention information. A summariser benefits from a graph because it can identify central nuclei and entity chains rather than treating every sentence as equally important.

A2.4 Three pitfalls

- A cue phrase in the middle of a sentence does not automatically create a new unit; splitting there can produce fragments without independent propositions.
- A forward scan answers “where was this entity last mentioned?” rather than “where will it next be mentioned?”.
- A flat sentence list hides long-distance relations, central claims and repeated entities; a graph supports coherent extraction and summarisation.

A3 – GENERATOR WITH FOCUS MEMORY (5 MARKS)

A3.1 Required data structures

Structure	Contents	Update rule
Focus list (k=2)	Up to two currently salient entity IDs, ordered by salience	Add mentioned entities; move current topic to front; evict lowest-ranked when full
Realisation descriptor	Last form, number, gender, semantic type, last mention	Update after each realised mention

	position	
Aggregation buffer	Compatible propositions awaiting a shared sentence	Flush when topic changes, relation is unsafe, or sentence limit is reached
Given/new flag	Whether entity has already been introduced in discourse	New entity uses an indefinite NP; known entity may use definite NP or pronoun

A3.2 Pseudocode

```
FUNCTION CHOOSE_FORM(entity e, focus F)
1. IF e is new THEN RETURN indefinite_full_NP(e)
2. IF e is in F AND no same-gender/number competitor exists
     AND antecedent is unambiguous THEN RETURN pronoun(e)
3. IF e is in F THEN RETURN definite_description(e)
4. IF a focus slot is free THEN insert e; RETURN definite_description(e)
5. Evict lowest-salience entity that is not needed by the next proposition
6. Insert e
7. RETURN definite_description(e)

FUNCTION AGGREGATE(p1, p2)
1. IF p1 and p2 share an agent/topic AND relation is safe
     THEN combine with a conjunction or subordinate clause
2. ELSE emit p1 and p2 as separate sentences
3. Never aggregate if it hides event order, cause or a change of agent
END
```

Selection rule: use the shortest referring expression that remains uniquely interpretable for the reader; prefer a pronoun only when its antecedent is in focus and no competing entity makes it ambiguous.

A3.3 Trace for F1–F4

Assumption: memory(server_1, 32 GB) introduces server_1; configure(ravi, server_1) introduces Ravi and refers to server_1; install(ravi, suite_1, server_1) introduces suite_1; crash(suite_1, during=build) continues the suite topic. Word counts below are transparent illustrative counts, counting whitespace-separated words.

Step	Realised sentence	Focus after step	Word count (running)
F1	A server has 32 GB of memory.	[server]	7
F2	Ravi configured the server.	[Ravi, server]	12
F3	He installed a compiler suite on it.	[suite, Ravi]	20
F4	The suite crashed during a build.	[suite, Ravi]	27

Naïve full-name baseline: “The server has 32 GB of memory. Ravi configured server_1. Ravi installed suite_1 on server_1. suite_1 crashed during build.” (about 30 whitespace tokens, depending on identifier expansion). The memory-based version is shorter and more natural, but it assumes the reader can identify the antecedents. A careful human may choose a definite description instead of “it” if two machines are salient.

A3.4 Failure case

Input propositions: “Ravi spoke to Arjun. He approved the deployment.” If both people are male and equally salient, a recency-based generator may choose “He” for Arjun even when Ravi was intended. The generator needs a hearer model and a confidence test; if two antecedents compete, repeat the name.

A4 – ENTITY–DISCOURSE GRAPH AND PARSER REPAIR (5 MARKS)

A4.1 Diagnose the weak algorithm

Text: (1) Ravi configured the lab server on Monday. (2) It crashed during the first build. (3) A second server arrived from the vendor on Tuesday. (4) Ravi configured it as well. (5) The room had become very warm by then.

Pair	Weak algorithm output	Problem / intended reading
(1)→(2)	ELABORATION because shared noun may be inferred	“It” refers to the first server; relation is Result/Sequence, not simple elaboration
(2)→(3)	SEQUENCE	Reasonable temporal sequence; new server is a distinct entity despite same head noun
(3)→(4)	ELABORATION due to shared “server”	“it” refers to the second server; repeated head noun does not mean same entity
(4)→(5)	SEQUENCE	May miss implicit causal/context relation: warm room may explain equipment behaviour, but evidence is insufficient to assert causation

The weak method also fails to represent distinct server_1 and server_2 as separate referents. Shared vocabulary is not proof of coreference.

A4.2 Graph design and pseudocode

Create one node per discourse referent, not one node per noun string. Every mention stores a pointer to an entity node. A new entity is created when the text introduces a distinct instance (“a second server”). A repeated mention links to an existing node only when syntax, discourse and identity evidence support it.

```
ALGORITHM BUILD_ENTITY_GRAPH(units)
1. Initialise ENTITY_NODES, MENTION_NODES and RELATIONS.
2. FOR each unit u in document order:
     extract noun phrases and pronouns
     FOR each mention m:
       IF m is an introduction (indefinite NP or explicit new-instance cue)
          create a new entity node e
       ELSE IF m is a pronoun or definite NP
          resolve against compatible active entities
          IF one candidate is reliable: link m to that entity
          ELSE store unresolved / ambiguous candidates
       store mention node m and its entity pointer
     infer relation to an open discourse unit using evidence
     update right frontier and salience
3. RETURN graph
END
```

A4.3 Demonstration and special cases

For the A4 text, create server_1 from (1), link "It" in (2) to server_1, create server_2 from "A second server" in (3), and link "it" in (4) to server_2. "Ravi" in (4) links to Ravi in (1). "The room" introduces room_1. Do not link the two server nodes merely because both have head noun "server".

For T, create server_1, old_machine_1, Ravi_1, toolchain_1, suite_1, fault_1, vendor_1 and module_1. Link "It" to server_1, "He" to Ravi_1, "it" to server_1, and "The suite" to suite_1.

Phenomenon	Example	Handling
Cataphora	"Before he spoke, Ravi checked the server."	Allow a forward candidate only inside a constrained clause; otherwise mark unresolved until later mention
One-anaphora	"A faster server arrived. Ravi configured a faster one."	Create a new server referent; inherit category/features, not identity
Implicit cause	"The memory filled. The suite crashed."	Label possible cause with confidence; do not assert if only temporal evidence exists
Same noun, different entity	"The first server… A second server…"	Create two entity nodes
Repeated full name	"Ravi configured it. Ravi then installed…"	Rewrite second occurrence as "He" only if uniquely resolvable

Safe rewrite rules: (1) replace a repeated full name with a pronoun only if one compatible antecedent is uniquely salient; (2) remove a sentence only if its proposition is entailed by another sentence and it adds no new temporal, causal or contrastive information.

A4.4 Challenge conditions

- Reported speech creates a separate speaker/source context. A pronoun inside a quotation must first be resolved within the quoted speaker's discourse space.
- A section heading can start a new discourse segment; deactivate old focus candidates unless an explicit link reintroduces them.
- Machine translation may lose grammatical gender. Gender must then be treated as unknown, not guessed; number, syntax and semantic compatibility may still be used.

The violated assumption is that all mentions share one flat narrator context and preserve the same agreement features. Nested speech, topic boundaries and translation loss require scoped discourse contexts and uncertainty-aware features.

A5 – COHERENCE AND GLOBAL SALIENCE (6 MARKS)

A5.1 Opportunities and safety conditions

Technique	Opportunity in T	Safety condition
Coreference chains	Track server, Ravi and compiler suite consistently	Only link when identity evidence is strong
Lexical chains	server–machine–lab–compiler–build indicate topic continuity	Do not equate related words with same entity
Centering	Choose "He" for Ravi when Ravi is the current centre	Avoid pronoun if another compatible person competes
Entity grid	Measure transitions of entities across sentences (subject/object/other/absent)	Use a suitable genre-specific baseline; grid score is not truth
Discourse extraction	Prefer central event "suite crashed" and its cause	Respect nucleus/satellite structure and preserve safety-critical facts
Distributional similarity	Recognise related words such as compiler/toolchain	Similarity indicates relatedness, not coreference

A5.2 GEN/KILL fixed-point computation

For forward salience propagation, GEN[u] contains entities made salient by unit u; KILL[u] contains entities displaced or made inactive. IN[u] is the union of predecessor OUT sets; OUT[u] = GEN[u] ∪ (IN[u] − KILL[u]). Initialise all sets empty, process units in document order, and repeat until no set changes. At a join, use union because an entity salient on either incoming path may remain a candidate, then apply agreement and segment filters during resolution.

```
ALGORITHM FORWARD_SALIENCE(G)
1. FOR each unit u: IN[u] ← ∅; OUT[u] ← ∅
2. REPEAT
     changed ← FALSE
     FOR each unit u in document order:
        INnew ← UNION of OUT[p] for every predecessor p of u
```

```
        OUTnew ← GEN[u] UNION (INnew − KILL[u])
        IF INnew ≠ IN[u] OR OUTnew ≠ OUT[u]:
            IN[u] ← INnew; OUT[u] ← OUTnew; changed ← TRUE
    UNTIL changed = FALSE
3. RETURN IN, OUT
END
```

Termination: the entity universe is finite, and each update only adds/removes members within finite sets; a monotone work-list implementation reaches a fixed point. For a simple linear report, one forward pass is sufficient; iterations are needed for branching graphs or cyclic discourse structures.

A5.3 Illustrative trace

Unit	GEN	KILL	IN	OUT
U1	server	—	∅	{server}
U2	old_machine	—	{server}	{server, old_machine}
U3	Ravi, server	—	{server, old_machine}	{server, old_machine, Ravi}
U4	suite	—	{server, old_machine, Ravi}	{server, old_machine, Ravi, suite}
U5	fault	—	{server, old_machine, Ravi, suite}	{server, old_machine, Ravi, suite, fault}
U6	vendor, module	old_machine	{server, old_machine, Ravi, suite, fault}	{server, Ravi, suite, fault, vendor, module}

Under this simplified linear graph, convergence occurs after one complete forward pass because each unit has one predecessor and no backward dependency. A second pass makes no changes. The table is illustrative; a production system should use mention-level salience decay and segment-specific KILL rules.

A5.4 Backward variant

To estimate presupposed or already-available information from the reader's perspective, propagate backward from later units. At a join, intersection is the conservative operation: an entity is considered required/available only if supported on all relevant continuations. Initialise at the document end with the empty set (or explicitly declared shared knowledge), then iterate backwards. This avoids assuming the reader knows everything mentioned on only one branch. The cost is conservatism: a summariser or translator may omit useful but branch-specific context.

PART B – IMPLEMENTATION AND COMPUTATIONAL INVESTIGATION

Implementation note: the assignment requires actual instrumentation. The tables below include reproducible test cases and illustrative expected outputs so the design can be checked. They are not claimed as measured results from a large external corpus. Run the appendix program on your own test set and replace illustrative values before submission.

B1 – IMPLEMENT AND INSTRUMENT THE DISCOURSE ANALYSER (7 MARKS)

B1.1 Validation tests

Test	Input summary	Gold expectation	Purpose
T1 – topic shift	Paragraph 1 about server; paragraph 2 about library	Pronoun in each paragraph resolves locally; old topic is deactivated	Checks paragraph boundary and topic reset
T2 – same head noun	"A server failed. A second server arrived. It was configured."	"It" → second server	Checks entity identity, not noun-string matching
T3 – boundary	"The compiler is ready."	No pronoun links	Checks zero-markable case without division-by-zero

Implementation modifications: sentence splitting is performed before mention extraction; paragraph breaks lower the salience of old entities; pleonastic "it" is excluded by a small rule list ("it is raining", "it seems"); and low-confidence pronouns remain unresolved. These modifications prevent false links that a purely recency-based pseudocode would produce.

B1.2–B1.4 Strategy comparison

Version 1 chooses the most recent compatible antecedent. Version 2 uses a weighted score: subject role +3, recent mention +2, repeated topic +2, parallel grammatical role +1, semantic compatibility +3; incompatible number/person is a hard rejection. This rule is preferable because grammatical role and semantic type often outweigh raw distance.

k	Strategy	Resolved / unresolved	Illustrative P/R/F1	Mean candidate set	Interpretation
1	V1	3 / 1	0.75 / 0.75 / 0.75	1.2	Very small focus causes missed older antecedents
1	V2	3 / 1	0.80 / 0.75 / 0.77	1.1	Better ranking but cannot recover evicted entity
2	V1	4 / 0	0.75 / 0.75 / 0.75	1.8	Recent-only makes one

					wrong choice
2	V2	4 / 0	1.00 / 1.00 / 1.00	1.8	Role and semantics correct the ambiguous choice
3	V1	4 / 0	0.75 / 0.75 / 0.75	2.0	Larger focus does not fix ranking
3	V2	4 / 0	1.00 / 1.00 / 1.00	2.0	Ranking still helps
Unlimited	V1	4 / 0	0.75 / 0.75 / 0.75	2.5	More candidates can increase confusion
Unlimited	V2	4 / 0	1.00 / 1.00 / 1.00	2.5	Candidate ranking remains important

The values above are an example reporting format, not benchmark claims. In a real run, calculate TP/FP/FN against the gold links for each k. The difference may disappear only when the candidate set contains a single compatible entity; increasing k alone does not solve ambiguity.

B2 – ENGINEER AN ADVERSARIAL INPUT (5 MARKS)

Use fixed focus size k=2. Both texts below contain six sentences and exactly four pronouns.

Text A – easy resolution

- (1) Maya entered the lab.
- (2) She opened the cabinet.
- (3) Maya removed a sensor.
- (4) She tested the sensor.
- (5) Maya recorded the result.
- (6) She left the lab.

Gold pronouns: She(2)→Maya; She(4)→Maya; She(6)→Maya; “the sensor” is a definite NP, not counted as a pronoun. To make the required four pronouns exact, treat “it” in the revised sentence below as the fourth pronoun: replace sentence (5) with “Maya recorded it.” The final Text A is therefore: (1) Maya entered the lab. (2) She opened the cabinet. (3) Maya removed a sensor. (4) She tested the sensor. (5) Maya recorded it. (6) She left the lab.

Text B – deliberately ambiguous

- (1) Arjun met Kiran near the lab.
- (2) He said the server was ready.

- (3) The technician called Arjun after lunch.
- (4) He told him it had failed.
- (5) Kiran checked the server with Arjun.
- (6) He said it could be repaired.

Text B has four pronouns: He(2), He(4), him(4), He(6), plus "it" appears twice; for a strict four-pronoun comparison, use "the server" in (4) and (6) instead of "it". Prediction: Text B performs worse because two male entities compete, "he/him" can refer to either person, and repeated server mentions do not guarantee the same machine. First likely error is He in sentence (2), because both Arjun and Kiran are compatible.

Measure	Text A – predicted	Text B – predicted
Pronouns	4	4 (after replacing extra "it" forms with "the server")
Correct links	4/4	1–2/4
Unresolved / ambiguous	0	1–2
Mean candidate set	1	2 or more
Reason	One salient compatible person	Competing same-gender candidates and topic shifts

Reordering can change results without changing facts: introduce the intended antecedent immediately before its pronoun and the candidate set becomes easier; insert another same-gender person just before the pronoun and ambiguity increases.

B3 – DIALOGUE AND CONVERSATIONAL AGENTS (8 MARKS)

B3.1 Dialog-act labels and three test dialogues

Use labels: REQUEST (asks for an action), ASK_INFO (requests information), INFORM (provides a fact), CONFIRM (accepts a proposal), CLARIFY (asks to disambiguate), REPAIR (corrects an earlier misunderstanding), THANK (social closing), OFFER (agent proposes an action).

Dialogue / turn	Utterance	Gold act	Predicted act	Appropriate response
Information U1	I need to renew three books.	REQUEST	REQUEST	Ask for library ID
Information A1	Could you give me your library ID?	ASK_INFO	ASK_INFO	Wait for ID
Information U2	It is 2024CS118.	INFORM	INFORM	Store ID
Clarification U1	Can you renew my book?	REQUEST	REQUEST	Ask which book / account
Clarification A1	Which book do you mean?	CLARIFY	CLARIFY	Request missing detail
Clarification U2	The NLP book.	INFORM	INFORM	Use NLP title

Repair U1	Renew the AI book.	REQUEST	REQUEST	Select AI book
Repair A1	I found the NLP book.	INFORM (wrong entity)	INFORM	Potential error
Repair U2	No, I said AI, not NLP.	REPAIR	REPAIR	Correct selected title

For the supplied dialogue D: U1=REQUEST; A1=ASK_INFO; U2=INFORM; A2=INFORM+OFFER; U3=CONFIRM+ASK_INFO; A3=INFORM+COMMITMENT; U4=THANK. The agent should store the ID, renewal request, Friday due date, two-week acceptance, NLP availability query and reservation-until-tomorrow commitment.

B3.2 Conservative and unsafe cases

Conservative case: "Would the NLP book be available tomorrow?" is a question, not proof that the book is available or reserved. A state tracker that marks every mentioned entity as established would incorrectly answer "Your book is reserved." Correct behaviour is to store an availability query as pending and wait for a verified catalogue result.

Unsafe transformation: merging "I can reserve it" with "I have reserved it" changes an offer into a completed commitment. The agent must preserve dialog-act force and completion status; it must not drop the confirmation turn.

B3.3 Scalability and state representations

Version 1 rebuilds state from the complete dialogue history on every turn. Version 2 maintains an incremental stack of salient acts and entities, updating only the current turn and evicting stale details under explicit rules. Let T be turns and E be distinct entities. Full-history rebuilding costs approximately $O(T^2+TE)$ across all turns; incremental updates cost $O(T+E)$ for bounded feature updates, excluding external retrieval.

Turns	**Entities**	**Acts**	**V1 operations (illustrative)**	**V2 operations (illustrative)**
10	5	10	150	35
50	20	50	3,750	170
100	40	100	15,000	340
500	150	500	375,000	1,650

The operation counts are illustrative estimates from a simple feature model, not measured wall-clock timings. Instrument actual feature comparisons and state updates in code. Assume a 100 ms response budget for dialogue-state work; use incremental state for long conversations and retain a compact audit history for recovery.

B3.4 Engineering trade-off

Incremental state reduces repeated work and makes the current task easier to retrieve. Its risk is premature eviction: a book title or user constraint may be needed later. Keep durable facts (ID, selected

title, confirmed action) in persistent slots; keep transient dialog acts in a bounded recency stack. Do not discard commitments or unresolved questions merely because they are old.

B4 – FINAL ENGINEERING DECISION: TRANSLATION AND ANSWERING SERVICE (5 MARKS)

B4.1 Three processing levels

Level	Processing order	When to use	Expected quality / cost
Level 0 – Direct	Sentence split → direct translation → answer	Low-risk, short notices; no cross-sentence references	Lowest latency; pronoun ambiguity remains
Level 1 – Fast	Light entity detection → local reference resolution → translation → surface check	Default for most notices	Moderate cost; resolves clear local references
Level 2 – Full	Segmentation → entity graph → discourse relations → global reference resolution → discourse plan → target-language realisation → translation verification	Long, safety-critical or ambiguous documents	Highest cost; best context handling, but may over-analyse

Reference resolution must precede translation when the source pronoun is ambiguous or target-language agreement requires an explicit antecedent. Discourse planning must precede surface realisation because the generator needs to know what information is central and what can safely be aggregated.

Quality reporting: evaluate on a held-out parallel set using BLEU or chrF and human adequacy. BLEU/chrF measures surface similarity, not clinical safety; human reviewers must judge whether entities, negation, quantities, dates and commitments are preserved. Do not invent BLEU scores without running a translation system and a reference set.

B4.2 Counter-example

A short notice contains two adjacent sentences with no pronouns: “The lab opens at 9 AM. Wear safety glasses.” Full document-level analysis adds cost but cannot improve the translation. It may even combine the sentences into an overly long Hindi sentence, reducing readability. The fast level is better for this input.

B4.3 Production decision note

- Default: Level 1 Fast. It provides local entity and pronoun handling without paying the full document-analysis cost on every request.

- On request or when risk triggers fire: Level 2 Full for long documents, unresolved pronouns, cross-paragraph references, legal/medical/safety instructions or translation ambiguity.
- Guarantees: preserve detected names, numbers, dates, negation and confirmed commitments; flag unresolved references. It does not guarantee the most natural translation or perfect discourse interpretation.
- Second-language cost: use an interlingua/meaning representation to separate source analysis from target-language generation. Add Hindi and Tamil realisation modules rather than duplicating the full analyser. Tamil's lower parallel-data availability makes rule/lexicon support and human review especially important.
- Skip or limit global analysis for short, self-contained notices with no referring expressions or discourse dependencies.
- Monitor reference-link precision/recall, unresolved rate, human adequacy, negation/number preservation, latency, correction rate and language-specific failure patterns before changing the default.

CONCLUSION

The recommended architecture separates interpretation from generation. The discourse analyser creates entity identities, mention links and discourse relations; the generator uses a bounded focus model to produce concise but unambiguous text; the dialogue manager stores confirmed facts and commitments; and the translation service operates on a meaning representation when cross-sentence context matters. A conservative system that admits uncertainty is safer than one that always produces a confident but unsupported answer.

APPENDIX – PYTHON REFERENCE IMPLEMENTATION

The following standard-library Python example demonstrates sentence splitting, simple mention extraction, candidate ranking, precision/recall/F1 and basic dialog-act classification. It is a teaching baseline, not a production-grade NLP system. Extend the lexicons and evaluate against manually annotated data.

```
import re
from collections import defaultdict

def split_sentences(text):
    return [s.strip() for s in re.split(r"(?<=[.!?])\\s+", text) if s.strip()]

PRONOUNS = {"he", "she", "it", "they", "him", "her", "them"}
MALE_NAMES = {"ravi", "arjun", "kiran"}
FEMALE_NAMES = {"maya"}

def extract_mentions(sentence, index):
    words = re.findall(r"\\b[A-Za-z]+\\b", sentence)
    found = []
    for w in words:
```

```
        low = w.lower()
        if low in PRONOUNS:
            found.append({"text": w, "kind": "pronoun", "sent": index})
        elif low in MALE_NAMES | FEMALE_NAMES:
            found.append({"text": w, "kind": "name", "sent": index})
    return found

def resolve_pronoun(pronoun, entities):
    p = pronoun.lower()
    candidates = []
    for e in entities:
        if p in {"he", "him"} and e["gender"] == "male":
            candidates.append(e)
        elif p in {"she", "her"} and e["gender"] == "female":
            candidates.append(e)
        elif p == "it" and e["type"] in {"object", "machine", "software"}:
            candidates.append(e)
        elif p == "they" and e["number"] == "plural":
            candidates.append(e)
    if not candidates:
        return None
    # Most recent compatible entity is the baseline strategy.
    return max(candidates, key=lambda e: e["last_seen"])

def score(gold, predicted):
    gold, predicted = set(gold), set(predicted)
    tp = len(gold & predicted)
    fp = len(predicted - gold)
    fn = len(gold - predicted)
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    return {"TP": tp, "FP": fp, "FN": fn,
            "precision": precision, "recall": recall, "F1": f1}

def classify_dialog_act(text):
    t = text.lower().strip()
    if any(x in t for x in ["no, ", "i meant", "not ", "correction"]):
        return "REPAIR"
    if "?" in t or t.startswith(("could you", "would you", "can you")):
        return "ASK_INFO"
    if any(x in t for x in ["thank you", "thanks"]):
        return "THANK"
    if t.startswith(("yes", "sure", "please do")):
```

```
        return "CONFIRM"
    if t.startswith(("i need", "please", "renew", "book")):
        return "REQUEST"
    return "INFORM"

if __name__ == "__main__":
    text = ("The department bought a new server. "
            "Ravi configured the server. He installed the compiler.")
    for i, sentence in enumerate(split_sentences(text), 1):
        print(i, sentence)
    print(score(
        gold={("He", "Ravi")},
        predicted={("He", "Ravi")}
    ))
    print(classify_dialog_act("I need to renew three books."))
```